

COMIC CRAFT

AI Comic Story Creator using Gemini Models

Full Project Explanation • Planning • Complete Code • Code Explanation • Proposed Solution • Core Features

Project Type	Generative AI / Web Application
Primary AI	Google Gemini Models
Prepared For	Project Documentation & Implementation
Prepared By	A. KISHORE
Document Flow	Input PDFs → Planning → Build → Explain → Proposed Solution

Goal: build a beginner-friendly AI web app that converts a user's comic idea into a structured story with title, characters, scenes, dialogue, visual prompts, and downloadable output.

1. Project Journey and Document Structure

This report follows the same sequence requested for the project: first it converts the supplied planning templates into Comic Craft project content; next it gives a complete working implementation and explains the code; finally it presents the proposed solution overview and core features.

Stage	What this document contains
1. Input / Planning	Brainstorming & idea prioritization, problem statements, and empathy map adapted to Comic Craft.
2. Project Design	Objective, users, workflow, architecture, technology stack, data flow, and UI plan.
3. Full Code	Flask backend, Gemini integration, HTML interface, CSS styling, JavaScript, requirements, and environment setup.
4. Code Explanation	Simple explanation of every important file, function, request, and response.
5. Final Solution	Proposed solution overview, core features, testing, limitations, future scope, and conclusion.

Source basis: the uploaded templates define three early project-planning activities: brainstorming/idea prioritization, customer problem statements, and a Say/Think/Do/Feel empathy map. This PDF fills those structures with content for Comic Craft.

Project Abstract

Comic Craft is an AI-powered comic story creation web application. A user enters a story idea, selects a genre, tone, target audience, number of scenes, and optional character details. The application sends a structured prompt to a Gemini model. Gemini returns a comic-ready story plan containing a title, synopsis, character profiles, scene-by-scene narration, dialogue, captions, and image-generation prompts. The result is displayed in a readable comic storyboard format and can be copied or saved.

2. Brainstorming & Idea Prioritization - Comic Craft

The supplied brainstorming template asks team members to list ideas without judging them first, then rate grouped ideas on feasibility and importance before selecting the final idea.

S.No	Team Member	Idea / Suggestion	Category	Group
1	A. KISHORE	AI generates a full comic story from one short idea	Generative AI	G1
2	A. KISHORE	Create characters with names, personality and roles	Story Design	G1
3	A. KISHORE	Generate scene-by-scene dialogue and captions	Comic Writing	G1
4	A. KISHORE	Create visual prompts for every comic panel	Image Prompting	G2
5	A. KISHORE	Genre, tone, audience and scene controls	Personalization	G3
6	A. KISHORE	Copy/download final story for later use	Productivity	G3

Idea Prioritization

Group	Final Idea	Feasibility	Importance	Priority	Selected
G1	Gemini-powered comic story + character + dialogue generator	High	High	1	Yes
G2	Panel-wise visual prompt generator	High	High	2	Yes
G3	Customization and export controls	High	Medium	3	Yes

Final Prioritized Idea

Comic Craft - AI Comic Story Creator using Gemini Models: a web application that transforms a simple user idea into a structured, creative, panel-ready comic story. The chosen scope is feasible for a student project because the application can use Gemini for text generation while the web application manages inputs, prompt construction, result display, and export.

3. Define Problem Statements - Comic Craft

The supplied problem-statement template uses the structure: I am → I'm trying to → But → Because → Which makes me feel. The following statements apply that structure to Comic Craft.

PS	I am (Customer)	I'm trying to	But	Because	Which makes me
PS-1	A student / beginner	Create an interesting comic story	struggle to organize plot and story	lack of experience	Stuck and less confident
PS-2	A content creator	Turn an idea into comic	Writing dialogue and panel takes time	Manual scripting is repetitive	Slow and frustrated
PS-3	A visual storyteller	Create consistent scenes	Descriptions vary between panels	repeated visual details	Uncertain about consistency

Main Problem Statement

Creative beginners often have good story ideas but find it difficult to convert those ideas into a complete comic structure with characters, scene progression, narration, dialogue, and visual descriptions. Manual creation can be slow and requires both storytelling and prompt-writing skills. Comic Craft addresses this by guiding the user through simple inputs and using Gemini to produce a structured first draft that the user can review and edit.

4. Empathy Map - Comic Craft

The supplied empathy-map template asks the team to understand the user from four perspectives: what the user Says, Thinks, Does, and Feels.

Persona	Student / Beginner Comic Creator
SAYS	"I have an idea but I don't know how to make it a full comic." "I want good dialogue quickly." "I need scene descriptions for each panel."
THINKS	Will my story be interesting? How do I keep characters consistent? Can AI save time without removing my control?
DOES	Writes short story ideas. Searches for inspiration. Edits AI output and uses it for comic creation.
FEELS	Excited about the idea. Confused about structure. Satisfied when a clear storyboard is generated.

User Need Derived from the Empathy Map

The user needs a simple tool that reduces the blank-page problem, provides a clear structure, and still keeps the user in control. Therefore, the interface should avoid complicated AI settings and instead use familiar choices such as genre, tone, audience, number of scenes, and optional character details.

5. Project Objectives and Scope

Primary Objective

To design and implement an AI-powered web application that uses Gemini models to transform a user-provided idea into a complete comic story blueprint.

Specific Objectives

- Collect a story idea and creative preferences through a clean web form.
- Generate a title, synopsis, characters, scene plan, narration, dialogue, captions, and visual prompts.
- Return the AI output in predictable JSON so the application can display it clearly.
- Provide error handling for missing API keys, invalid input, model errors, and malformed AI responses.
- Allow the generated story to be copied or downloaded as a text file.
- Keep the project easy to run locally in VS Code for a student demonstration.

Project Scope

Version 1 focuses on text-based comic planning. It does not require automatic image generation. Instead, every scene contains a visual prompt that can later be sent to an image model. This keeps the first implementation simple, testable, and suitable for a project demonstration.

6. Proposed System Workflow

Step	Process	Output
1	User enters idea and options	Story requirements
2	Frontend validates input	Clean request
3	Flask API receives JSON	Server-side data
4	Prompt builder creates detailed instruction	Gemini prompt
5	Gemini model generates structured JSON	Comic story data
6	Backend validates/parses response	Safe JSON response
7	Frontend renders story cards	Readable storyboard
8	User copies/downloads output	Reusable project content

Architecture

Browser UI → POST /api/generate → Flask Backend → Gemini API → structured JSON → Flask → Browser renderer.

The API key remains on the server and is not placed in frontend JavaScript. This is important because frontend keys can be exposed to users.

Technology Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript	Form, responsive interface, rendering and download
Backend	Python + Flask	API endpoint, prompt construction, validation
AI	Google Gemini via google-genai SDK	Story generation
Configuration	.env / environment variables	Secure API key and model name
Data Format	JSON	Reliable structured communication

7. Project Folder Structure

```
comic-craft/
├── app.py
├── requirements.txt
├── .env
├── .gitignore
├── templates/
│   └── index.html
└── static/
    ├── style.css
    └── script.js
```

Create these files exactly in VS Code. The .env file contains the Gemini API key and must not be uploaded to a public GitHub repository.

8. Full Code - requirements.txt

```
Flask>=3.0
google-genai>=1.0
python-dotenv>=1.0
```

Explanation: Flask runs the web server. google-genai provides the Gemini client. python-dotenv loads local environment variables from the .env file.

.env

```
GEMINI_API_KEY=PASTE_YOUR_GEMINI_API_KEY_HERE
GEMINI_MODEL=gemini-2.5-flash
```

The model name is configurable. If a different Gemini model is available in your account, change only GEMINI_MODEL rather than editing the Python code.

.gitignore

```
.env
__pycache__/*
*.pyc
.venv/
venv/
```

9. Full Code - app.py

```
import json
import os
import re

from dotenv import load_dotenv
from flask import Flask, jsonify, render_template, request
from google import genai

load_dotenv()

app = Flask(__name__)

API_KEY = os.getenv("GEMINI_API_KEY", "").strip()
MODEL_NAME = os.getenv("GEMINI_MODEL", "gemini-2.5-flash").strip()

client = genai.Client(api_key=API_KEY) if API_KEY else None

def clamp(value, minimum, maximum, default):
    try:
        number = int(value)
        return max(minimum, min(maximum, number))
    except (TypeError, ValueError):
        return default

def clean_text(value, max_length=2000):
    if value is None:
        return ""
    value = re.sub(r"\s+", " ", str(value)).strip()
    return value[:max_length]

def build_prompt(data):
    idea = clean_text(data.get("idea"), 2500)
    genre = clean_text(data.get("genre"), 80) or "Adventure"
    tone = clean_text(data.get("tone"), 80) or "Cinematic"
    audience = clean_text(data.get("audience"), 80) or "General"
    character = clean_text(data.get("character"), 500) or "Create suitable original characters."
    scenes = clamp(data.get("scenes"), 3, 12, 6)

    return f"""
You are Comic Craft, an expert comic writer and storyboard planner.

Create an ORIGINAL comic story from the user's idea.

USER INPUT
Idea: {idea}
Genre: {genre}
Tone: {tone}
Audience: {audience}
Character notes: {character}
Number of scenes: {scenes}

RULES
1. Keep the story appropriate for the selected audience.
2. Maintain character consistency from beginning to end.
3. Make every scene visually clear and useful for comic panels.
4. Use concise, natural dialogue.
5. Do not copy existing copyrighted characters or stories.
6. Return ONLY valid JSON. Do not use markdown fences.

Return exactly this JSON structure:
{{
  "title": "string",
  "tagline": "string",
  "synopsis": "string",
  "genre": "string",
  "characters": [
    {{
      "name": "string",
      "role": "string",
      "personality": "string",
      "appearance": "string"
    }}
  ],
  "scenes": [
    {{
      "scene_number": 1,
      "scene_title": "string",
```

```

        "location": "string",
        "narration": "string",
        "visual_prompt": "string",
        "dialogue": [
            {
                "speaker": "string",
                "line": "string"
            }
        ],
        "caption": "string"
    }
},
"ending": "string",
"moral_or_theme": "string"
}
"""

def extract_json(text):
    text = text.strip()
    text = re.sub(r"^```(?:json)?\s*", "", text, flags=re.I)
    text = re.sub(r"\s*```$", "", text)

    try:
        return json.loads(text)
    except json.JSONDecodeError:
        start = text.find("{")
        end = text.rfind("}")
        if start != -1 and end != -1 and end > start:
            return json.loads(text[start:end + 1])
        raise

@app.get("/")
def home():
    return render_template("index.html")

@app.post("/api/generate")
def generate_comic():
    if client is None:
        return jsonify({
            "error": "Gemini API key is missing. Add GEMINI_API_KEY to your .env file."
        }), 500

    data = request.get_json(silent=True) or {}
    idea = clean_text(data.get("idea"), 2500)

    if len(idea) < 10:
        return jsonify({
            "error": "Please enter a story idea with at least 10 characters."
        }), 400

    prompt = build_prompt(data)

    try:
        response = client.models.generate_content(
            model=MODEL_NAME,
            contents=prompt
        )

        raw_text = (response.text or "").strip()
        if not raw_text:
            raise ValueError("The model returned an empty response.")

        comic = extract_json(raw_text)

        if not isinstance(comic, dict):
            raise ValueError("Unexpected AI response format.")

        return jsonify({
            "success": True,
            "comic": comic
        })

    except Exception as exc:
        app.logger.exception("Comic generation failed")
        return jsonify({
            "error": "Comic generation failed.",
            "details": str(exc)
        }), 500

if __name__ == "__main__":

```

```
app.run(debug=True)
```

10. app.py - Simple Code Explanation

Part	What it does
load_dotenv()	Reads GEMINI_API_KEY and GEMINI_MODEL from the .env file.
genai.Client(...)	Creates the Gemini API client only when an API key exists.
clean_text()	Removes extra spaces and limits input length.
clamp()	Keeps the requested scene count between 3 and 12.
build_prompt()	Converts user choices into a strict instruction for the Gemini model.
extract_json()	Parses Gemini output and also handles accidental ```json code fences.
GET /	Loads the main web page.
POST /api/generate	Validates the idea, calls Gemini, parses the result and returns JSON.
try/except	Prevents the server from crashing and sends a readable error to the UI.

Why JSON output is used

A plain paragraph from an AI model is difficult for a website to separate into characters, scenes, dialogue and captions. JSON gives each item a clear field name, allowing JavaScript to render a professional storyboard automatically.

11. Full Code - templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Comic Craft - AI Comic Story Creator</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <header class="hero">
    <div class="nav">
      <div class="brand">COMIC CRAFT</div>
      <div class="badge">Powered by Gemini</div>
    </div>

    <div class="hero-content">
      <p class="eyebrow">AI COMIC STORY CREATOR</p>
      <h1>Turn one idea into a complete comic story.</h1>
      <p class="sub">
        Generate characters, scenes, dialogue, captions and visual prompts
        in seconds.
      </p>
    </div>
  </header>

  <main class="container">
    <section class="panel">
      <h2>Create your comic</h2>

      <form id="comicForm">
        <label for="idea">Story idea</label>
        <textarea id="idea" required minlength="10"
          placeholder="Example: A shy student discovers a robot hidden inside the college library..."></textarea>

        <div class="grid">
          <div>
            <label for="genre">Genre</label>
            <select id="genre">
              <option>Adventure</option>
              <option>Fantasy</option>
              <option>Sci-Fi</option>
              <option>Comedy</option>
              <option>Mystery</option>
              <option>Superhero</option>
              <option>Educational</option>
            </select>
          </div>

          <div>
            <label for="tone">Tone</label>
            <select id="tone">
              <option>Cinematic</option>
              <option>Funny</option>
              <option>Emotional</option>
              <option>Dark</option>
              <option>Inspirational</option>
              <option>Light-hearted</option>
            </select>
          </div>

          <div>
            <label for="audience">Audience</label>
            <select id="audience">
              <option>General</option>
              <option>Kids</option>
              <option>Teens</option>
              <option>Young Adults</option>
            </select>
          </div>

          <div>
            <label for="scenes">Scenes</label>
            <input id="scenes" type="number" min="3" max="12" value="6">
          </div>
        </div>

        <label for="character">Character notes (optional)</label>
        <input id="character"
          placeholder="Example: Hero: Arjun, 19, curious, wears a blue hoodie">
      </form>
    </section>
  </main>
</body>
</html>
```

```

        <button id="generateBtn" type="submit">Generate Comic Story</button>
    </form>

    <div id="status" class="status"></div>
</section>

<section id="result" class="result hidden">
    <div class="result-head">
        <div>
            <p class="eyebrow">GENERATED STORY</p>
            <h2 id="comicTitle"></h2>
            <p id="tagline"></p>
        </div>
        <div class="actions">
            <button id="copyBtn" class="secondary">Copy</button>
            <button id="downloadBtn" class="secondary">Download</button>
        </div>
    </div>

    <div id="summaryCard" class="summary-card"></div>

    <h3>Characters</h3>
    <div id="characters" class="cards"></div>

    <h3>Comic Scenes</h3>
    <div id="scenesList" class="scene-list"></div>

    <div id="endingCard" class="ending-card"></div>
</section>
</main>

<footer>
    Comic Craft • AI Comic Story Creator using Gemini Models
</footer>

<script src="/static/script.js"></script>
</body>
</html>

```

12. index.html - Simple Code Explanation

The HTML file creates the visible application. It contains a hero section, an input form, dropdown controls, a generate button, a status message, and an initially hidden result area.

- `#comicForm` collects the story idea and user preferences.
- `#status` shows messages such as “Generating...” or an error.
- `#result` stays hidden until the backend returns a successful comic.
- `#characters` and `#scenesList` are empty containers filled dynamically by JavaScript.
- Copy and Download buttons let the user reuse the generated story outside the app.

13. Full Code - static/style.css

```
:root {
  --bg: #070b16;
  --panel: #111827;
  --card: #151d31;
  --text: #f8fafc;
  --muted: #94a3b8;
  --line: #26324b;
  --purple: #8b5cf6;
  --cyan: #22d3ee;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: Inter, Arial, sans-serif;
  background: linear-gradient(180deg, #070b16, #0b1020);
  color: var(--text);
  min-height: 100vh;
}

.hero {
  border-bottom: 1px solid var(--line);
  background:
    radial-gradient(circle at 15% 20%, #7c3aed55, transparent 30%),
    radial-gradient(circle at 85% 10%, #0891b255, transparent 28%);
}

.nav, .hero-content, .container {
  width: min(1100px, 92%);
  margin: auto;
}

.nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 22px 0;
}

.brand {
  font-weight: 900;
  letter-spacing: 2px;
}

.badge {
  color: var(--cyan);
  border: 1px solid #22d3ee55;
  padding: 8px 12px;
  border-radius: 999px;
  font-size: 13px;
}

.hero-content {
  padding: 55px 0 70px;
}

.eyebrow {
  color: var(--cyan);
  font-size: 12px;
  font-weight: 800;
  letter-spacing: 2px;
}

h1 {
  max-width: 800px;
  font-size: clamp(38px, 7vw, 72px);
  line-height: 1;
  margin: 10px 0 18px;
}

.sub {
  max-width: 650px;
  color: var(--muted);
  font-size: 18px;
  line-height: 1.6;
}

.container { padding: 40px 0 80px; }
```

```

.panel, .result {
  background: #0f172acc;
  border: 1px solid var(--line);
  border-radius: 24px;
  padding: 28px;
  box-shadow: 0 20px 70px #00000035;
}

.result { margin-top: 28px; }

label {
  display: block;
  margin: 16px 0 8px;
  font-weight: 700;
}

textarea, input, select {
  width: 100%;
  background: #0b1222;
  color: var(--text);
  border: 1px solid var(--line);
  border-radius: 12px;
  padding: 13px 14px;
  outline: none;
}

textarea {
  min-height: 150px;
  resize: vertical;
}

textarea:focus, input:focus, select:focus {
  border-color: var(--purple);
}

.grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 14px;
}

button {
  margin-top: 20px;
  border: 0;
  border-radius: 12px;
  padding: 13px 18px;
  font-weight: 800;
  cursor: pointer;
  background: linear-gradient(90deg, var(--purple), #2563eb);
  color: white;
}

button:disabled { opacity: .55; cursor: not-allowed; }

.secondary {
  margin: 0 0 0 8px;
  background: transparent;
  border: 1px solid var(--line);
}

.status { min-height: 24px; margin-top: 14px; color: var(--muted); }
.status.error { color: #fca5a5; }

.hidden { display: none; }

.result-head {
  display: flex;
  justify-content: space-between;
  gap: 20px;
  align-items: start;
}

.summary-card, .ending-card, .character-card, .scene {
  background: var(--card);
  border: 1px solid var(--line);
  border-radius: 16px;
  padding: 18px;
}

.cards {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 14px;
}

```

```

.scene-list { display: grid; gap: 16px; }

.scene {
  border-left: 4px solid var(--purple);
}

.scene h4 { margin: 0 0 8px; font-size: 20px; }

.meta, .visual, .caption {
  color: var(--muted);
  line-height: 1.6;
}

.dialogue {
  margin: 12px 0;
  padding: 10px 12px;
  background: #0b1222;
  border-radius: 10px;
}

footer {
  text-align: center;
  color: var(--muted);
  padding: 30px;
  border-top: 1px solid var(--line);
}

@media (max-width: 700px) {
  .grid, .cards { grid-template-columns: 1fr; }
  .result-head { flex-direction: column; }
  .actions { display: flex; }
}

```

14. style.css - Simple Code Explanation

The CSS creates a modern dark comic/AI interface using a navy background, purple and cyan accents, rounded cards, responsive grids, and mobile breakpoints. The design is intentionally lightweight and does not depend on an external CSS framework.

- `:root` stores reusable colors.
- `.grid` displays form controls in two columns on larger screens.
- `.cards` creates a character-card grid.
- `.scene` visually separates each generated comic scene.
- `@media` changes grids to one column on smaller mobile screens.

15. Full Code - static/script.js

```
const form = document.getElementById("comicForm");
const statusBox = document.getElementById("status");
const result = document.getElementById("result");
const generateBtn = document.getElementById("generateBtn");

let latestComic = null;

function escapeHtml(value = "") {
  return String(value)
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;")
    .replaceAll("'", "&quot;")
    .replaceAll('"', "&#039;");
}

function setStatus(message, isError = false) {
  statusBox.textContent = message;
  statusBox.className = isError ? "status error" : "status";
}

function renderComic(comic) {
  latestComic = comic;

  document.getElementById("comicTitle").textContent =
    comic.title || "Untitled Comic";

  document.getElementById("tagline").textContent =
    comic.tagline || "";

  document.getElementById("summaryCard").innerHTML = `
    <h3>Synopsis</h3>
    <p>${escapeHtml(comic.synopsis || "")}</p>
    <p class="meta"><strong>Genre:</strong>
      ${escapeHtml(comic.genre || "")}</p>
  `;

  const characters = Array.isArray(comic.characters)
    ? comic.characters : [];

  document.getElementById("characters").innerHTML =
    characters.map(character => `
    <article class="character-card">
      <h4>${escapeHtml(character.name || "Character")}</h4>
      <p><strong>Role:</strong>
        ${escapeHtml(character.role || "")}</p>
      <p><strong>Personality:</strong>
        ${escapeHtml(character.personality || "")}</p>
      <p><strong>Appearance:</strong>
        ${escapeHtml(character.appearance || "")}</p>
    </article>
  `).join("");

  const scenes = Array.isArray(comic.scenes) ? comic.scenes : [];

  document.getElementById("scenesList").innerHTML =
    scenes.map(scene => {
      const dialogue = Array.isArray(scene.dialogue)
        ? scene.dialogue : [];

      return `
      <article class="scene">
        <p class="eyebrow">SCENE
          ${escapeHtml(scene.scene_number || "")}</p>
        <h4>${escapeHtml(scene.scene_title || "")}</h4>
        <p class="meta"><strong>Location:</strong>
          ${escapeHtml(scene.location || "")}</p>
        <p>${escapeHtml(scene.narration || "")}</p>

        <div class="visual">
          <strong>Visual Prompt:</strong><br>
          ${escapeHtml(scene.visual_prompt || "")}
        </div>

        ${dialogue.map(item => `
          <div class="dialogue">
            <strong>${escapeHtml(item.speaker || "Character")}</strong>
            ${escapeHtml(item.line || "")}
          </div>
        `).join("")}
      `;
    }).join("");
}
```

```

        <p class="caption"><strong>Caption:</strong>
        ${escapeHtml(scene.caption || "")}</p>
    </article>
    `;
    }).join("");

document.getElementById("endingCard").innerHTML = `
    <h3>Ending</h3>
    <p>${escapeHtml(comic.ending || "")}</p>
    <p><strong>Theme / Moral:</strong>
    ${escapeHtml(comic.moral_or_theme || "")}</p>
    `;

result.classList.remove("hidden");
result.scrollIntoView({ behavior: "smooth", block: "start" });
}

form.addEventListener("submit", async (event) => {
    event.preventDefault();

    const payload = {
        idea: document.getElementById("idea").value.trim(),
        genre: document.getElementById("genre").value,
        tone: document.getElementById("tone").value,
        audience: document.getElementById("audience").value,
        scenes: Number(document.getElementById("scenes").value),
        character: document.getElementById("character").value.trim()
    };

    if (payload.idea.length < 10) {
        setStatus("Please enter a longer story idea.", true);
        return;
    }

    generateBtn.disabled = true;
    generateBtn.textContent = "Generating...";
    setStatus("Gemini is building your comic story...");

    try {
        const response = await fetch("/api/generate", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(payload)
        });

        const data = await response.json();

        if (!response.ok) {
            throw new Error(data.details || data.error || "Request failed");
        }

        renderComic(data.comic);
        setStatus("Comic story generated successfully.");
    } catch (error) {
        setStatus(error.message, true);
    } finally {
        generateBtn.disabled = false;
        generateBtn.textContent = "Generate Comic Story";
    }
});

document.getElementById("copyBtn").addEventListener("click", async () => {
    if (!latestComic) return;
    await navigator.clipboard.writeText(
        JSON.stringify(latestComic, null, 2)
    );
    setStatus("Comic JSON copied to clipboard.");
});

document.getElementById("downloadBtn").addEventListener("click", () => {
    if (!latestComic) return;

    const blob = new Blob(
        [JSON.stringify(latestComic, null, 2)],
        { type: "application/json" }
    );

    const url = URL.createObjectURL(blob);
    const link = document.createElement("a");
    link.href = url;
    link.download = "comic-craft-story.json";
    link.click();
    URL.revokeObjectURL(url);
});

```

16. script.js - Simple Code Explanation

Function / Block	Purpose
escapeHtml()	Prevents generated text from being treated as HTML code when inserted into the page.
setStatus()	Displays progress and error messages.
renderComic()	Reads the returned JSON and builds title, characters, scenes, dialogue and ending cards.
form submit listener	Collects input, sends a POST request to /api/generate, and waits for the response.
fetch()	Connects the browser to the Flask backend.
Copy button	Copies the complete generated JSON.
Download button	Creates a JSON file in the browser and downloads it.

Important Security Detail

The frontend never contains the Gemini API key. The browser talks only to the Flask backend. The backend uses the secret key to call Gemini. This separation is safer than placing the key inside script.js.

17. How to Run the Project in VS Code

Step	Command / Action
1	Create the comic-craft folder and files shown in the folder structure.
2	Open the folder in VS Code.
3	Open Terminal → New Terminal.
4	Create environment: <code>python -m venv .venv</code>
5	Windows activate: <code>.venv\Scripts\activate</code>
6	Install packages: <code>pip install -r requirements.txt</code>
7	Add your Gemini key to <code>.env</code> .
8	Run: <code>python app.py</code>
9	Open the local address shown by Flask, normally <code>http://127.0.0.1:5000</code>
10	Enter an idea and click Generate Comic Story.

Example Test Input

Idea: A shy college student discovers a tiny robot inside an old library book. The robot can predict small events, but every prediction creates a funny new problem.

Genre: Sci-Fi Tone: Funny Audience: Teens Scenes: 6

18. Expected AI Output Structure

```
{
  "title": "The Library Bot",
  "tagline": "Every prediction has a surprise.",
  "synopsis": "...",
  "genre": "Sci-Fi Comedy",
  "characters": [
    {
      "name": "Arun",
      "role": "Main character",
      "personality": "Curious but shy",
      "appearance": "College student with a blue backpack"
    }
  ],
  "scenes": [
    {
      "scene_number": 1,
      "scene_title": "The Strange Book",
      "location": "College library",
      "narration": "...",
      "visual_prompt": "...",
      "dialogue": [
        {"speaker": "Arun", "line": "What is that sound?"}
      ],
      "caption": "Some books contain more than stories."
    }
  ],
  "ending": "...",
  "moral_or_theme": "..."
}
```

Actual story wording will vary because generative AI is non-deterministic. The important requirement is that the response follows the same JSON fields so the interface can render it.

19. Testing Plan

Test	Input / Condition	Expected Result
Valid generation	Normal story idea	Story renders with title, characters and scenes
Short input	Less than 10 characters	Validation message; no API call
Missing API key	No GEMINI_API_KEY	Readable server error
Scene minimum	1 scene	Backend clamps to 3
Scene maximum	30 scenes	Backend clamps to 12
Mobile UI	Screen under 700px	One-column layout
Special characters	<, >, &, quotes in AI text	Displayed safely through escaping
Model failure	Network/API error	UI shows error and button is re-enabled
Malformed JSON	Model returns invalid format	Backend catches parse failure

Success Criteria

- The user can create a story without writing code.
- The output contains consistent structured sections.
- No API key is exposed in frontend files.
- The interface works on laptop and mobile widths.
- Errors are understandable and do not crash the application.

20. Proposed Solution Overview

Comic Craft proposes a guided AI co-creation workflow rather than a completely automatic black-box creator. The user provides the creative direction; the application structures that direction into a strong prompt; Gemini generates a first draft; and the user remains responsible for reviewing, editing, and deciding how to use the result.

The solution directly addresses the planning problems identified earlier: difficulty expanding a small idea, time spent creating dialogue, uncertainty about scene structure, and difficulty writing consistent visual descriptions. The application reduces those tasks to a short form and returns a reusable comic blueprint.

Core Features

Feature	Description
Idea-to-Comic Generation	Converts a short concept into a complete structured comic story.
Genre & Tone Control	Lets the user choose adventure, fantasy, sci-fi, comedy, mystery and other styles.
Audience Control	Guides the model toward age-appropriate wording and story intensity.
Character Builder	Creates names, roles, personality traits and appearance descriptions.
Scene-by-Scene Storyboard	Breaks the plot into ordered scenes with locations and narration.
Dialogue Generator	Creates concise character dialogue for each scene.
Panel Visual Prompts	Provides image-ready descriptions that can later connect to an image model.
Captions & Ending	Adds comic captions, ending text and a theme/moral.
Copy & Download	Allows the generated result to be reused in other tools.
Responsive Interface	Works across laptop, tablet and mobile screen sizes.

21. Advantages, Limitations and Future Scope

Advantages

- Reduces the time needed to create a first comic draft.
- Helps beginners who understand ideas better than story structure.
- Produces structured output instead of one long AI paragraph.
- Separates secret API configuration from frontend code.
- Can be extended into a larger multimodal comic production platform.

Current Limitations

- AI output can vary between runs and may still need human editing.
- Character consistency is textual in Version 1; generated images would require additional controls.
- The app depends on internet access and Gemini API availability.
- Very complex stories may need multiple generation/editing stages rather than one request.

Future Enhancements

- Generate actual comic panel images from each visual prompt.
- Add character reference images and visual consistency controls.
- Add user login, project history and cloud saving.
- Add edit/regenerate buttons for individual scenes instead of regenerating the whole story.
- Export to PDF comic storyboard, DOCX script, or presentation.
- Support Tamil and other languages for input and generated dialogue.
- Add speech bubbles, panel layout templates and drag-and-drop comic page design.

22. Final Conclusion

Comic Craft - AI Comic Story Creator using Gemini Models is a practical Generative AI project that combines creative writing, structured prompting, web development, and API integration. The project begins with user-centered planning through brainstorming, problem statements, and empathy mapping; moves into a clear technical architecture; and ends with a working implementation that can generate structured comic stories from simple user ideas.

The strongest design decision is to keep the user in control while using Gemini as a creative assistant. The system does not replace the creator's decisions; it accelerates the first draft and provides an organized foundation that can be edited, illustrated, and expanded.

Final project outcome: a complete student-ready web application concept with planning documentation, source code, code explanation, testing plan, proposed solution overview, and core features.

Submission Checklist

- Planning content completed for Comic Craft
- Problem statement completed
- Empathy map completed
- Folder structure prepared
- Gemini API key configured through .env
- Backend and frontend code added
- Local testing completed
- Screenshots can be added to the final college submission if required
- GitHub upload should exclude .env